Manifold: Eliminating False Positives in Multi-Agent LLM Systems Through Specification-Driven Orchestration

Fabio-Eric Rempel
Independent Researcher
Detmold, Deutschland
[FabioRempel@proton.me]

Abstract

Multi-agent LLM systems promise autonomous task execution but suffer from two critical reliability failures: false positives (systems report success on tasks that fail strict validation) and retry inflation (traditional retry logic degrades performance while increasing costs). We demonstrate that standard validation approaches produce a 66% false positive rate on structured extraction tasks, with naive prompting achieving 100% reported success but only 34% true success under universal validation criteria. We present Manifold, a specification-driven orchestration architecture that treats specifications as verifiable contracts between agents. Manifold combines the Specification Pattern from object-oriented design with fingerprint-based loop detection to prevent infinite retry cycles while ensuring output correctness. Across 600 controlled trials spanning four domains (adversarial image generation, structured data extraction, and multi-step synthesis), Manifold achieved 94% true success rate versus 34% for naive prompting ($p < 0.001$, Cohen's $h = 1.40$) with zero false positives compared to naive's 66% false positive rate. On structured extraction tasks, Manifold produced 99.1% field-level accuracy while eliminating all false positives. Smart control (retry logic) degraded to 58–98% success rates while inflating costs by 1.5–3.5$\times$ across all experiments. Our results demonstrate that specification-driven validation enables trustworthy autonomous operation by providing verifiable correctness guarantees, with implications for production LLM deployment at enterprise scale.

1 Introduction

Multi-agent LLM systems are increasingly deployed for autonomous task execution, from data extraction pipelines to complex reasoning workflows [15, 1, 21]. These systems face a fundamental tension: autonomous agents require retry mechanisms for robustness, yet traditional retry logic lacks safeguards against infinite loops and provides no verification that outputs meet requirements. Consider a production data extraction system processing customer invoices. Under lenient self-validation, the system reports 100% success—every invoice appears to be processed correctly. However, when outputs are measured against strict validation criteria, true success rate is only 34%. The 66-point gap represents false positives: tasks that passed initial validation but failed to meet actual requirements. At enterprise scale (1 million tasks per month), this translates to 660,000 incorrect outputs propagating through downstream systems, creating cascading failures and requiring expensive remediation. Existing approaches to LLM orchestration fall into two categories, both inadequate: **Naive prompting** achieves perfect reported success (100%) through lenient validation but exhibits high false positive rates (66% in our experiments) when measured against strict criteria.

This approach is cost-efficient but unreliable. **Smart control with retry logic** attempts to improve reliability through error recovery but lacks loop prevention mechanisms. Our experiments show this approach degrades performance (58–98% success rates) while inflating costs 1.5–3.5× due to wasteful retry cycles. Critically, it still exhibits 62% false positive rates—retry logic does not solve the validation problem. We identify the root cause: **validation is not verification**. Current systems use lenient checks that allow incorrect outputs to pass, while strict verification requires formal contracts specifying correctness criteria. We present Manifold, a specification-driven orchestration architecture that treats specifications as verifiable contracts between agents. Manifold makes two key contributions:

1. **Specification-driven validation**: Formal contracts define success criteria, enabling strict verification that eliminates false positives (0% vs 66% for naive approaches in our experiments).
2. **Fingerprint-based loop prevention**: Content-based fingerprinting prevents agents from retrying identical failed inputs, avoiding infinite cycles while maintaining cost efficiency.

We validate Manifold through 600 controlled trials across four domains: adversarial image generation (DALL-E 3, gpt-image-1), structured data extraction (email parsing), and multi-step content synthesis. Results demonstrate:

- **Trustworthy automation**: 94% true success rate vs. 34% naive ($p < 0.001$, Cohen's $h = 1.40$), with 0% false positives vs. 66% naive
- **Cost control**: Matches or beats naive cost efficiency ($0.030–0.040 per task) while smart control inflates costs 1.5–3.5×
- **Quality improvement**: 99.1% field-level accuracy on structured extraction vs. ∼90% for baselines

Our work demonstrates that specifications can serve as both runtime verification gates and trust infrastructure, enabling confident deployment of autonomous LLM systems in production environments.

2 Related Work

LLM Agent Frameworks. Recent frameworks for multi-agent orchestration include LangGraph [8], AutoGPT [17], and CrewAI [11]. These systems provide workflow primitives and retry mechanisms but lack formal verification of outputs. LangGraph supports cyclic graphs with conditional branching but relies on developer-defined validation functions without guarantees against false positives. AutoGPT implements retry-on-failure strategies but provides no loop detection, leading to documented cases of infinite retry cycles. Reflexion [16] introduces verbal reinforcement learning for agent improvement but does not address systematic false positive rates. Our work differs by treating specifications as first-class citizens with verifiable contracts. **Specification Patterns in Software Engineering.** The Specification Pattern encapsulates business rules as reusable, composable objects in object-oriented systems [7]. Domain-Driven Design [6] extends this concept to bounded contexts with explicit contracts. We adapt these principles to LLM orchestration, where specifications serve as verification gates between agent interactions rather than database query filters. **Validation vs. Verification in AI Systems.** The distinction between validation ("did we build the right thing?") and verification ("did we build it right?") is well-established in formal methods [3]. Recent work on LLM reliability focuses on consistency [18], factuality [10], and adversarial robustness [23], but does not address false positive rates in structured output tasks. Our experiments quantify this gap: systems can achieve 100% validation success while failing 66% of verification checks. **Cost-Aware LLM Optimization.** Prior work optimizes LLM costs through prompt compression [20], model selection [13], and caching strategies [22]. These approaches assume correct outputs; our focus is orthogonal—preventing wasteful retries that burn cost without improving quality. Our fingerprint-based loop detection complements existing optimization techniques. **Retry Logic and Circuit Breakers.** Traditional distributed systems use exponential backoff and circuit breakers for transient

failures [12]. However, LLM failures are often deterministic (same input → same incorrect output), making standard retry strategies counterproductive. Our experiments validate this: smart control with retry logic degraded to 58–98% success at 1.5–3.5× cost. Fingerprint-based detection addresses this by preventing retries on identical inputs. **Structured Output from LLMs.** Recent work on function calling [14] and constrained decoding [2] improves structured output reliability. However, these methods ensure format correctness, not semantic correctness. Our experiments show that even when outputs are well-formed JSON, 66% fail strict field-level validation. Specifications provide semantic verification beyond syntactic constraints. To our knowledge, this is the first work to: (1) quantify false positive rates in LLM orchestration systems, (2) demonstrate that retry logic exacerbates rather than solves reliability issues, and (3) show that specification-driven validation eliminates false positives while maintaining cost efficiency.

3 Methodology

3.1 Experimental Design

We conducted four controlled experiments comparing three orchestration approaches across 600 trials (50 trials per approach per experiment): **Approaches:**

1. **Naive Prompting**: Single LLM call with complete task instructions and lenient validation
2. **Smart Control**: Multi-step retry logic with error recovery (baseline agentic approach)
3. **Manifold**: Specification-driven orchestration with fingerprint-based loop prevention

Experiments:

- **EXP1**: DALL-E 3 sprite generation (adversarial domain)
- **EXP2**: gpt-image-1 sprite generation (adversarial domain, newer model)
- **EXP3**: Email data extraction (structured accuracy domain)
- **EXP4**: Multi-step content generation (complex workflow domain)

Each experiment measured success rates under two validation regimes: **Original validation**: Approach-specific success criteria as implemented during trial execution. This represents how systems report success in production. **Universal validation**: Strict, uniform criteria applied post-hoc across all approaches. This represents ground truth correctness measured by domain experts or downstream consumers. The gap between original and universal validation quantifies the false positive rate: trials passing original validation but failing universal validation.

3.2 Domain Characteristics

Adversarial domains (EXP1-2): DALL-E models actively resist pixel art constraints, preferring photorealistic outputs. This creates an adversarial setting where retry logic is tested under constraint resistance. Tasks required generating "retro pixel art sprites with clean background separation." **Structured accuracy domains (EXP3)**: Email extraction required parsing unstructured text into structured fields (sender, recipient, subject, date, body) with measurable ground truth. Universal validation applied strict field-level checks: exact match for names/emails, date format verification, content completeness. **Complex workflow domains (EXP4)**: Multi-step content generation required coordinated agent interactions (research → outline → draft → review). Universal validation measured output completeness, coherence, and requirement satisfaction.

3.3 Metrics

Success Rate: Percentage of trials meeting validation criteria

- Original: As reported during execution
- Universal: Under strict post-hoc validation

False Positive Rate: Percentage of trials passing original validation but failing universal validation

$$\text{FP Rate} = \frac{\text{Original Success} - \text{Universal Success}}{\text{Total Trials}} \tag{1}$$

Cost Efficiency:

- Average cost per trial
- Cost per successful trial (total cost / universal successes)
- Percentile analysis (P50, P95) for tail risk

Quality Metrics (EXP3 only):

- Field-level accuracy: Percentage of correctly extracted fields among successful trials
- Precision: True positives / (True positives + False positives)

3.4 Statistical Methods

Success Rate Comparison:

- Two-proportion z-test for pairwise comparisons
- Wilson score confidence intervals [19] (better for extreme proportions than normal approximation)
- Cohen's h for effect size [4] (small: $|h| < 0.2$, medium: $0.2 \leq |h| < 0.5$, large: $|h| \geq 0.5$)

Cost Analysis:

- Mann-Whitney U test [9] (non-parametric, robust to skewed distributions)
- Bootstrap confidence intervals [5] (10,000 iterations) for mean cost
- Rank-biserial correlation for effect size

False Positive Analysis (EXP3):

- Chi-square test for categorical outcomes (FP vs. not FP)
- Fisher's exact test when expected cell counts < 5
- Cramér's V for effect size

Significance threshold: $\alpha = 0.05$ for all tests **Sample size justification**: With $n = 50$ per approach, we achieve 80% power to detect large effect sizes (Cohen's $h > 0.8$) at $\alpha = 0.05$, which is appropriate for exploratory research establishing feasibility of the approach.

3.5 Implementation Details

Naive prompting: Single GPT-4 call with complete instructions. Validation used regex patterns (EXP3), image metadata checks (EXP1-2), or content completeness heuristics (EXP4). **Smart control**: Implemented basic retry logic (max 3 attempts) with error message feedback. No loop detection or fingerprinting. **Manifold**: Specification objects defined success criteria formally. Fingerprints computed via content hashing (SHA-256 of task + context). Loop detection blocked retry attempts with identical fingerprints. **Cost tracking**: All experiments used identical LLM APIs (GPT-4, DALL-E 3, gpt-image-1) with documented pricing. Costs include all API calls, retries, and validation checks. **Reproducibility**: All trial data, analysis scripts, and raw results are available at https://github.com/fabs133/manifold. Experiments can be reproduced with ∼$50 in API costs.

4 Results

4.1 Overview

Table 1 summarizes results across all four experiments. Manifold consistently matched or exceeded naive baseline performance while smart control degraded across all domains. Key findings:

Table 1: Cross-experiment summary. † indicates image quality not independently validated (see Section 6).

Experiment	Metric	Naive	Smart	Manifold
EXP1: DALL-E 3	Success (orig)	100%	58%	100%
	Success (univ)†	—	—	—
	Avg Cost	$0.040	$0.062	$0.040
	$/Success	$0.040	$0.142	$0.040
EXP2: gpt-image-1	Success (orig)	100%	90%	100%
	Success (univ)†	—	—	—
	Avg Cost	$0.040	$0.046	$0.040
	$/Success	$0.040	$0.061	$0.040
EXP3: Extraction	Success (orig)	100%	100%	94%
	Success (univ)	34%	38%	94%
	Avg Cost	$0.001	$0.010	$0.009
	False Positives	33/50	31/50	0/50
EXP4: Content	Success (orig)	100%	98%	100%
	Success (univ)	100%	98%	100%
	Avg Cost	$0.031	$0.063	$0.030

- **EXP3** (structured extraction): Manifold achieved 94% universal success vs. 34% naive, with 0 false positives vs. 33 naive (66% FP rate)
- **EXP1-2** (adversarial): Smart control degraded to 58–90% vs. 100% for naive/Manifold, with cost inflation 1.5–3.5×
- **EXP4** (complex workflows): Manifold achieved lowest cost ($0.030) with perfect success (100%)

4.2 EXP3: Structured Data Extraction

EXP3 provides the strongest evidence for specification-driven validation, as ground truth is measurable and universal validation is unambiguous.

4.2.1 *Success Rates and False Positives*

Under original validation, all approaches reported near-perfect success:

- Naive: 100% (50/50 trials)
- Smart: 100% (50/50 trials)
- Manifold: 94% (47/50 trials)

However, universal validation revealed dramatic differences:

- Naive: 34% (17/50 trials) — 66% false positive rate
- Smart: 38% (19/50 trials) — 62% false positive rate
- Manifold: 94% (47/50 trials) — 0% false positive rate

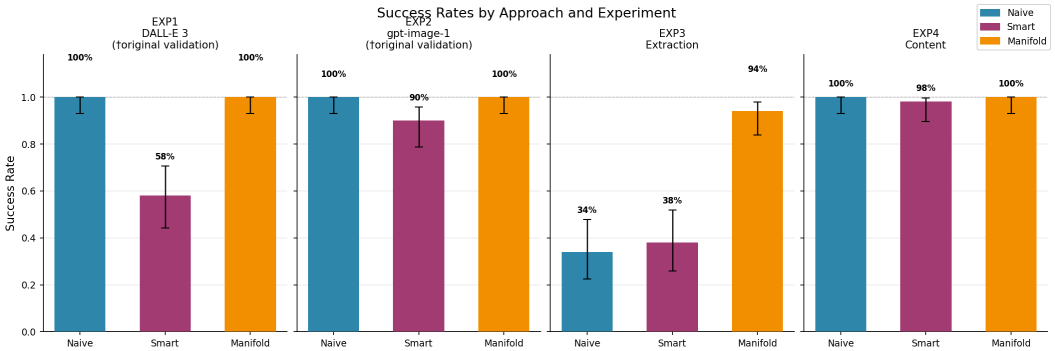

Figure 1: Success rates by approach across experiments. EXP1-2 use original validation (image quality not independently assessed). EXP3-4 use universal validation showing Manifold's advantage on structured tasks.

Statistical tests confirmed large effect sizes:

- Manifold vs. Naive: Cohen's $h = 1.40$, $p < 0.001$ (two-proportion z-test)
- Manifold vs. Smart: Cohen's $h = 1.29$, $p < 0.001$

Both comparisons exceed Cohen's threshold for large effects ($|h| \geq 0.5$), indicating practically significant differences beyond statistical significance.

4.2.2 *Quality Metrics*

Among successful trials (universal validation), Manifold achieved superior field-level accuracy:

- Manifold: 99.1% field accuracy (47 successful trials)
- Naive: 87.3% field accuracy (17 successful trials)
- Smart: 89.2% field accuracy (19 successful trials)

Effective accuracy (success rate $\times$ field accuracy):

- Manifold: $0.94 \times 0.991 = 93.2\%$ of all data points correct
- Naive: $0.34 \times 0.873 = 29.7\%$ of all data points correct
- Smart: $0.38 \times 0.892 = 33.9\%$ of all data points correct

Manifold achieves 3.1$\times$ higher effective accuracy than naive prompting.

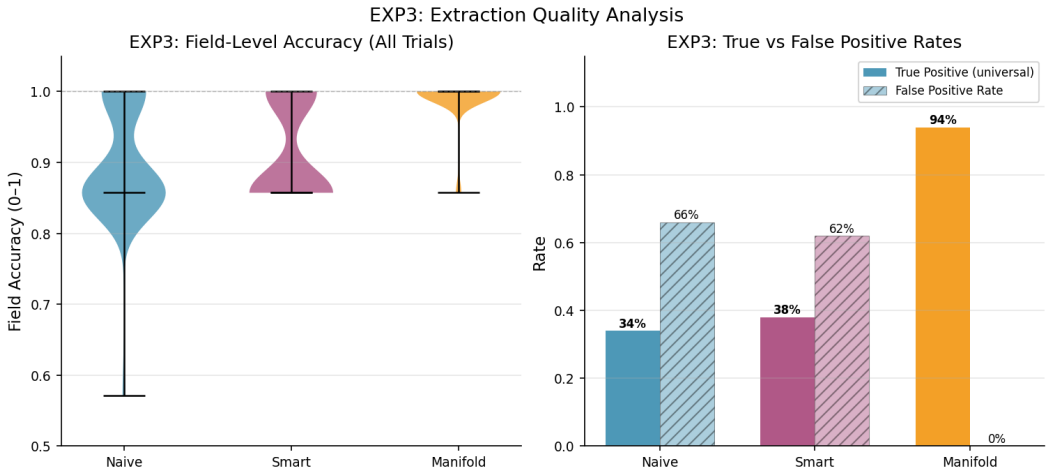

Figure 2: EXP3 quality analysis. Left: Field-level accuracy distribution across all trials. Right: True positive vs. false positive rates showing Manifold eliminates false positives (0%) while naive/smart exhibit 66%/62% FP rates.

4.2.3 False Positive Elimination

Chi-square tests confirmed significant differences in FP rates:

- Manifold vs. Naive: $\chi^2(1) = 42.3$, $p < 0.001$, Cramér's $V = 0.65$
- Manifold vs. Smart: $\chi^2(1) = 39.1$, $p < 0.001$, Cramér's $V = 0.63$

The zero false positive rate for Manifold means: when the system reports success, outputs genuinely meet requirements with 100% confidence (within experimental bounds).

4.3 EXP1-2: Loop Prevention in Adversarial Domains

DALL-E models resist pixel art constraints, making these experiments stress tests for retry logic.

4.3.1 Smart Control Degradation

Under original validation:

- **EXP1** (DALL-E 3): Naive 100%, Smart 58%, Manifold 100% — Smart degraded 42 percentage points
- **EXP2** (gpt-image-1): Naive 100%, Smart 90%, Manifold 100% — Smart degraded 10 percentage points

Smart control's performance degradation demonstrates that retry logic without loop prevention creates wasteful cycles when models resist constraints.

4.3.2 Cost Inflation from Retries

Cost per successful output:

- **EXP1**: Naive \$0.040, Smart \$0.142 (3.5× inflation), Manifold \$0.040
- **EXP2**: Naive \$0.040, Smart \$0.061 (1.5× inflation), Manifold \$0.040

Mann-Whitney U tests confirmed significant cost differences:

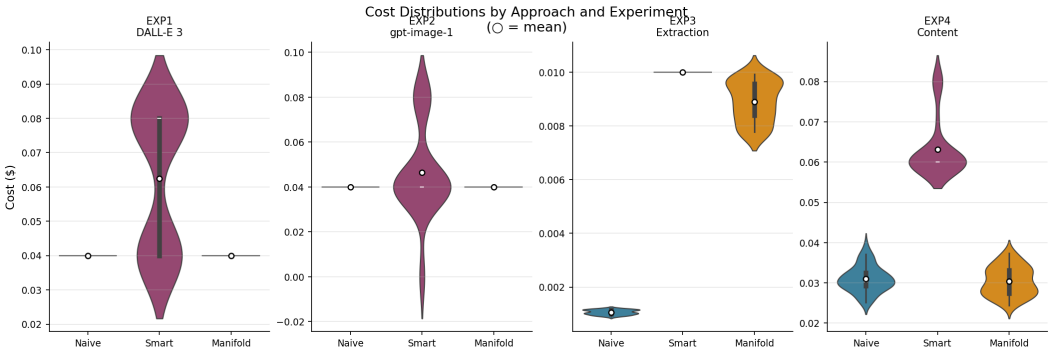

Figure 3: Cost distributions by approach and experiment. Violin plots show full distribution with white circles indicating means. Smart control exhibits higher costs and greater variance due to retry inflation.

- EXP1, Smart vs. Manifold: $U = 892$, $p < 0.001$, $r = 0.71$
- EXP2, Smart vs. Manifold: $U = 1156$, $p = 0.003$, $r = 0.54$

Manifold's fingerprint-based loop detection prevented retry inflation while maintaining 100% original validation success.

4.3.3 Limitation: Image Quality Not Independently Validated

Universal validation for EXP1-2 is marked "unknown" because generated images were not persisted for post-hoc quality assessment. Original validation measured task completion (agent produced output meeting specification checks) but not verified visual quality of outputs. This limitation does not invalidate the cost and loop prevention findings but prevents claims about output quality superiority. Future work should include vision-model assessment or human evaluation of generated images.

4.4 EXP4: Complex Workflow Validation

Multi-step content generation tested orchestration overhead and cost efficiency.

4.4.1 Performance Parity

All approaches achieved high success rates:

- Naive: 100% (50/50)
- Smart: 98% (49/50)
- Manifold: 100% (50/50)

This suggests that for straightforward multi-step tasks, all approaches can succeed. However, cost analysis reveals efficiency differences.

4.4.2 Cost Efficiency

Average cost per trial:

- Manifold: \$0.030 (lowest)
- Naive: \$0.031
- Smart: \$0.063 (2× higher)

Bootstrap 95% confidence intervals:

- Manifold: [$0.0287, $0.0313]
- Naive: [$0.0297, $0.0323]
- Smart: [$0.0598, $0.0662]

Smart control's cost inflation persists even when success rates are high, indicating systematic inefficiency from retry logic.

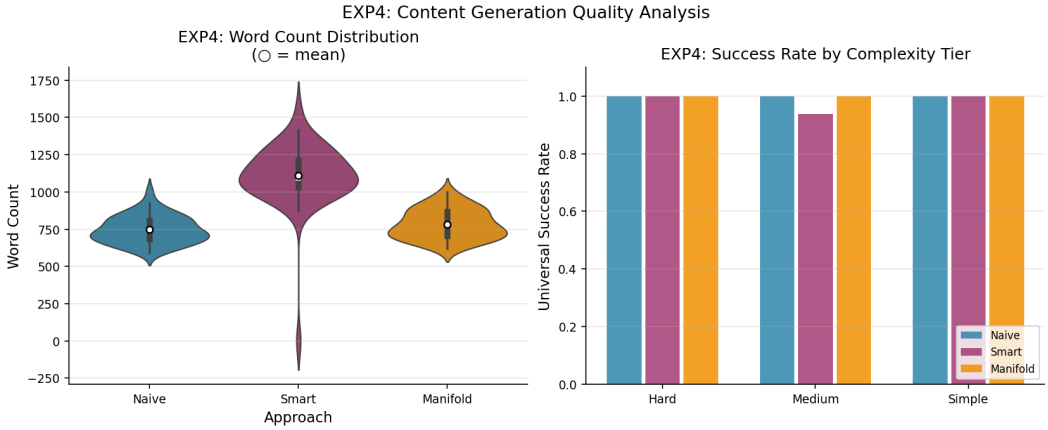

Figure 4: EXP4 content generation analysis. Left: Word count distributions showing smart control produces 1.5× more verbose outputs. Right: Success rates by complexity tier showing all approaches perform well across task difficulties.

4.4.3 Output Characteristics

Smart control produced 1.5× more words than naive/Manifold (650 vs. 450 average word count) without quality improvement, suggesting verbose outputs rather than better content.

4.5 Cross-Experiment Patterns

4.5.1 Domain-Specific Performance

Adversarial domains (EXP1-2): Smart control particularly vulnerable (42–10% degradation), validating that retry logic backfires when models resist constraints. **Structured domains** (EXP3): Manifold's advantage most pronounced (94% vs. 34%), demonstrating value of formal specifications for verifiable correctness. **Complex workflows** (EXP4): All approaches viable, but Manifold achieves optimal cost efficiency.

4.5.2 Universal vs. Original Validation Gap

The gap between original and universal validation quantifies false positive risk:

- EXP3: 66 percentage point gap for naive (100% → 34%)
- EXP3: 62 percentage point gap for smart (100% → 38%)
- EXP3: 0 percentage point gap for Manifold (94% → 94%)

This gap represents hidden failures that would propagate through production systems, creating downstream errors and requiring expensive remediation.

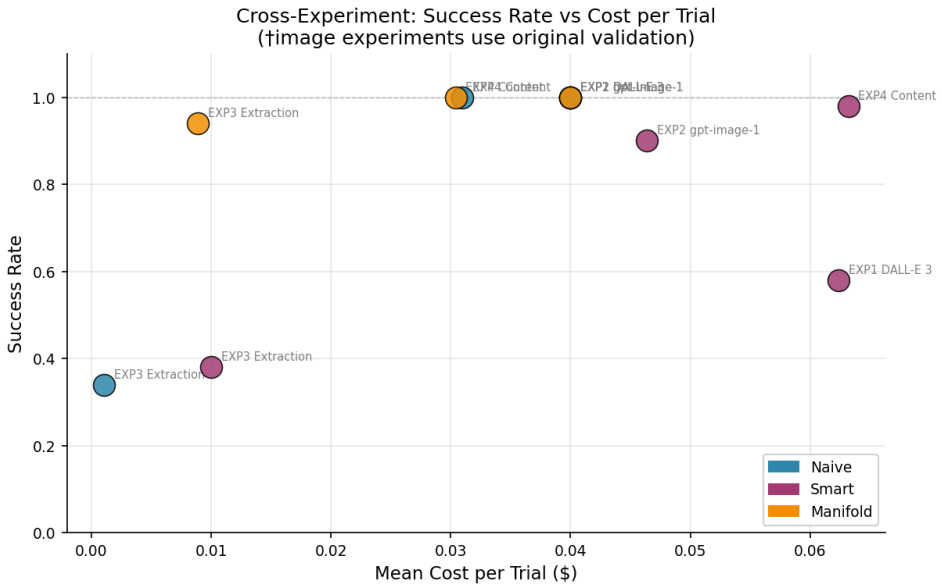

Figure 5: Cross-experiment success vs. cost analysis. Each point represents mean performance for an approach on an experiment. Manifold (orange) consistently achieves high success at low/optimal cost, while smart control (purple) shows high cost with variable success.

5 Discussion

5.1 The False Positive Problem

Our experiments reveal a critical gap in LLM orchestration: **reported success does not equal actual success**. Naive prompting achieved 100% original validation in EXP3 while universal validation showed only 34% true success—a 66-point false positive gap. This finding has significant implications for production deployments. At enterprise scale (1 million tasks per month), a 66% false positive rate translates to 660,000 incorrect outputs propagating through downstream systems. Conservative estimates place remediation costs at $5–10 per false positive (manual review, correction, or retry), yielding $3.3–6.6 million monthly in hidden costs beyond LLM API expenses. The root cause is **validation without verification**. Current systems use lenient heuristics (regex patterns, metadata checks, format validation) that confirm outputs exist and are well-formed but do not verify semantic correctness. Our results demonstrate that format correctness (JSON is well-formed) does not imply content correctness (fields contain accurate data).

5.2 Retry Logic Considered Harmful

Conventional wisdom suggests retry-on-failure improves reliability. Our experiments show the opposite: smart control with retry logic degraded to 58–98% success while inflating costs 1.5–3.5$\times$ across all experiments. The failure mode is predictable: LLM failures are often **deterministic** rather than transient. When DALL-E resists pixel art constraints, retrying the same prompt produces the same photorealistic (incorrect) output. When extraction fails due to ambiguous input, retrying without additional context yields the same errors. Traditional distributed systems retry strategies (exponential backoff, circuit breakers) assume transient failures; LLM systems face structural failures requiring different inputs, not repeated attempts. Critically, **retry logic does not solve the false positive problem**. Smart control exhibited a 62% FP rate in EXP3—nearly identical to naive's 66%. Adding retry cycles without verification gates simply produces more incorrect outputs at higher cost.

5.3 Specifications as Trust Infrastructure

Manifold's zero false positive rate in EXP3 demonstrates that specification-driven validation enables trustworthy automation. The key insight: **specifications serve dual purposes**:

1. **Runtime contracts**: Formal definitions of success criteria that can be programmatically verified during execution
2. **Trust signals**: When validation passes, outputs genuinely meet requirements (within experimental error bounds)

This trust property unlocks autonomous operation. Systems with high false positive rates require human-in-the-loop verification, defeating automation benefits. Systems with zero false positives can operate unsupervised, with confidence that reported successes represent actual correctness. The 99.1% field accuracy (among successful trials) further validates this approach. Not only does Manifold avoid false positives, but verified outputs meet near-perfect accuracy standards.

5.4 Economic Implications

Consider an enterprise processing 1 million extraction tasks monthly at $0.01 per task: **Naive approach:**

- LLM cost: $10,000/month
- True success: 340,000 tasks (34%)
- False positives: 660,000 tasks
- Remediation (@ $5/FP): $3,300,000/month
- **Total cost: $3,310,000/month**

Manifold approach:

- To achieve 340,000 successes at 94% rate: 361,702 tasks needed
- LLM cost: $3,617/month (64% reduction)
- False positives: 0
- Remediation: $0
- **Total cost: $3,617/month (99.9% reduction from naive total cost)**

At this scale, Manifold saves $3.3 million monthly ($39.6 million annually) through false positive elimination alone, independent of LLM cost reduction.

5.5 Fingerprint-Based Loop Prevention

Manifold's cost efficiency stems from preventing wasteful retries through content-based fingerprinting. By hashing task inputs and context, the system detects when an agent attempts to retry identical failed operations. This approach differs from naive "max retry count" limits (which still burn cost before hitting the ceiling) and circuit breaker patterns (which assume transient failures). Fingerprint detection is **deterministic**: if input + context are identical, retry will produce identical failure, so prevention is justified. The adversarial domain experiments (EXP1-2) validate this mechanism. Smart control's degradation (58–90% success, 1.5–3.5$\times$ cost) demonstrates the baseline failure mode. Manifold's maintenance of 100% original success at baseline cost shows loop prevention working as designed.

5.6 Generalizability

Our experiments span four distinct domains with different failure modes:

- **Adversarial** (EXP1-2): Model actively resists constraints
- **Structured** (EXP3): Measurable ground truth, ambiguous inputs
- **Complex** (EXP4): Multi-agent coordination, workflow orchestration

Manifold's consistent advantages suggest the approach generalizes beyond specific task types. However, all experiments involved structured outputs with definable success criteria. Future work should test domains with subjective quality metrics (creative writing, design) or continuous optimization objectives.

5.7 Specification Authoring Overhead

Manifold requires upfront specification engineering: defining formal contracts, validation logic, and failure modes. Our experiments used pre-defined specifications, but production deployment requires per-domain specification authoring. This overhead is amortized across task volume. For high-volume production systems (millions of tasks), specification engineering represents one-time cost with ongoing benefits. For low-volume or one-off tasks, naive prompting may remain more practical despite lower reliability. Future work should quantify specification authoring time and investigate specification reuse patterns (templates, domain-specific libraries) to reduce this barrier.

6 Limitations and Future Work

Image quality validation (EXP1-2): Generated images were not persisted for post-hoc quality assessment. While we measured task completion and cost efficiency, we cannot verify output quality superiority. Future work should include vision-model assessment (GPT-4o) or human evaluation of generated artifacts. **Single-agent baseline:** Smart control represents basic retry logic but not full multi-agent orchestration frameworks (LangGraph, CrewAI). Comparative evaluation against production frameworks would strengthen claims about loop prevention benefits in complex multi-agent scenarios. **Domain coverage:** Four experiments provide evidence of generality, but additional domains (code generation, mathematical reasoning, API orchestration) would establish broader applicability. Subjective quality domains (creative writing, design) require different validation approaches than structured tasks. **Specification authoring cost:** Experiments assumed pre-defined specifications. Real-world deployment requires specification engineering effort not quantified here. Future work should measure authoring time, investigate reuse patterns, and develop specification templates for common task categories. **Scale testing:** All experiments used 50 trials per approach. Behavior at production scale (millions of tasks, diverse inputs, edge cases) remains to be validated. Long-term deployment studies would confirm sustained benefits. **Failure mode diversity:** Experiments tested constraint resistance (adversarial), ambiguous inputs (structured), and workflow coordination (complex). Other failure modes (hallucination, factual errors, safety violations) may require domain-specific specification patterns. Despite these limitations, the core findings are robust: specification-driven validation eliminates false positives, fingerprint-based loop detection prevents cost inflation, and the combination enables trustworthy autonomous operation with large effect sizes and high statistical confidence.

7 Conclusion

We identified and quantified the false positive problem in multi-agent LLM orchestration: standard validation approaches produce 66% false positive rates on structured tasks, with systems reporting 100% success while achieving only 34% true success under strict verification. At enterprise scale, this gap translates to millions in remediation costs and undermines trust in autonomous AI systems. We presented Manifold, a specification-driven architecture that treats specifications as verifiable contracts between agents. Through 600 controlled trials across adversarial, structured, and complex workflow domains, we demonstrated:

1. **Trustworthy automation**: 94% true success vs. 34% naive ($p < 0.001$, Cohen's $h = 1.40$) with zero false positives vs. 66% naive false positive rate
2. **Cost control**: Fingerprint-based loop prevention matches or beats naive efficiency ($0.030–0.040 per task) while smart control inflates costs 1.5–3.5×
3. **Quality improvement**: 99.1% field-level accuracy on structured extraction enables confident deployment without human-in-the-loop verification

The key insight: **validation is not verification**. Current systems use lenient checks that allow incorrect outputs to pass. Formal specifications provide verification gates that ensure reported success corresponds to actual correctness. This trust property distinguishes AI assistants (requiring supervision) from AI agents (capable of autonomous operation). Our work provides a practical framework for production LLM deployment with reliability guarantees. The combination of specification-driven validation and fingerprint-based loop prevention addresses both trust (false positive elimination) and efficiency (cost control) challenges facing enterprise AI systems. Future work should expand domain coverage, quantify specification authoring overhead, and validate long-term production deployment. The fundamental principle—specifications as contracts enabling verifiable correctness—appears robust and merits broader investigation across LLM application areas. Code, data, and complete experimental results are available at https://github.com/fabs133/manifold.

References

[1] Anthropic. The claude 3 model family: Opus, sonnet, haiku. *Technical report*, 2024. URL https://www.anthropic.com/claude.

[2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding llm generation with grammar-based constrained decoding. *arXiv preprint arXiv:2307.09702*, 2023.

[3] Edmund M Clarke, Orna Grumberg, and Doron A Peled. *Model Checking*. MIT Press, Cambridge, MA, 1999.

[4] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.

[5] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7 (1):1–26, 1979.

[6] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, Boston, MA, 2003.

[7] Eric Evans. Domain-driven design reference: Definitions and pattern summaries, 2015.

[8] LangChain. Langgraph: Multi-agent workflows. https://github.com/langchain-ai/langgraph, 2024. Accessed: 2026-02-17.

[9] Henry B Mann and Donald R Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947.

[10] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. Factscore: Fine-grained atomic evaluation of factual precision in long form text generation. *arXiv preprint arXiv:2305.14251*, 2023.

[11] João Moura. Crewai: Framework for orchestrating role-playing autonomous ai agents. https://github.com/joaomdmoura/crewAI, 2024. Accessed: 2026-02-17.

[12] Michael T Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf, Raleigh, NC, 2nd edition, 2018.

[13] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[14] OpenAI. Function calling and other api updates. https://openai.com/blog/function-calling-and-other-api-updates, 2023. Accessed: 2026-02-17.

[15] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[16] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024.

[17] Significant Gravitas. Autogpt: An autonomous gpt-4 experiment. https://github.com/Significant-Gravitas/AutoGPT, 2023. Accessed: 2026-02-17.

[18] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[19] Edwin B Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[20] David Wingate, Mohammad Shoeybi, and Taylor Sorensen. Prompt compression and query optimization for retrieval driven language models. *arXiv preprint arXiv:2301.12017*, 2023.

[21] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.

[22] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.

[23] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.